```
"""
Bot d'alerte memecoins : détecte les tokens dont le profil DexScreener vient
d'être créé/payé (image + liens) et les "call" sur Telegram dès que tes critères
sont réunis.

Installation :  pip install aiohttp
Variables d'environnement :
    TELEGRAM_TOKEN    = token donné par @BotFather
    TELEGRAM_CHAT_ID  = id du chat / groupe / canal où envoyer les calls
Lancement :  python dex_paid_alert_bot.py
"""
import asyncio
import json
import logging
import os
import time

import aiohttp

# ====================== CONFIG (modifie ici) ======================
# Slugs de chaînes DexScreener. Vérifie celui de Robinhood Chain dans l'URL
# d'une paire sur dexscreener.com (dexscreener.com/<slug>/0x...).
CHAINS = {"ethereum", "bsc", "robinhood"}

MIN_VOLUME_USD = 10_000       # volume 24h minimum
MIN_MCAP_USD = 30_000
MAX_MCAP_USD = 200_000
MIN_LIQUIDITY_USD = 10_000
MAX_PAIR_AGE_HOURS = 24       # "newest" : paire créée il y a moins de X heures
REQUIRE_PROFILE = True        # image + au moins un lien (site ou réseau social)

PROFILE_POLL_SECONDS = 1.2    # limite API profils : 60 req/min -> garde >= 1.1
RECHECK_SECONDS = 8           # re-test des tokens profilés qui ne passent pas en
WATCH_EXPIRY_HOURS = 6        # on arrête de surveiller un token après X heures
# ==================================================================

API = "https://api.dexscreener.com"
TG_TOKEN = os.getenv("TELEGRAM_TOKEN", "")
TG_CHAT = os.getenv("TELEGRAM_CHAT_ID", "")
ALERTED_FILE = "alerted.json"

logging.basicConfig(level=logging.INFO, format="%(asctime)s %(message)s")
log = logging.getLogger("bot")
```

```
seen_profiles: set[str] = set()                     # "chain:address" déjà vus
watchlist: dict[str, tuple[str, str, float]] = {}  # key -> (chain, addr, ajouté_
alerted: set[str] = set()
unknown_chains_logged: set[str] = set()


def load_alerted():
    try:
        with open(ALERTED_FILE) as f:
            alerted.update(json.load(f))
    except Exception:
        pass


def save_alerted():
    try:
        with open(ALERTED_FILE, "w") as f:
            json.dump(sorted(alerted), f)
    except Exception as e:
        log.warning("save_alerted: %s", e)


async def get_json(session: aiohttp.ClientSession, url: str):
    try:
        async with session.get(url, timeout=aiohttp.ClientTimeout(total=8)) as r:
            if r.status == 429:
                log.warning("Rate limit atteint, pause 5s")
                await asyncio.sleep(5)
                return None
            if r.status != 200:
                log.warning("HTTP %s sur %s", r.status, url)
                return None
            return await r.json()
    except Exception as e:
        log.warning("Erreur réseau %s : %s", url, e)
        return None


def num(x) -> float:
    try:
        return float(x or 0)
    except (TypeError, ValueError):
        return 0.0


def evaluate(pair: dict) -> tuple[bool, str]:
    """Retourne (ok, raison_du_refus)."""
```

```
    liq = num((pair.get("liquidity") or {}).get("usd"))
    vol = num((pair.get("volume") or {}).get("h24"))
    mcap = num(pair.get("marketCap")) or num(pair.get("fdv"))
    created = num(pair.get("pairCreatedAt"))
    info = pair.get("info") or {}

    if liq < MIN_LIQUIDITY_USD:
        return False, f"liq {liq:.0f}"
    if vol < MIN_VOLUME_USD:
        return False, f"vol {vol:.0f}"
    if not (MIN_MCAP_USD <= mcap <= MAX_MCAP_USD):
        return False, f"mcap {mcap:.0f}"
    if created and (time.time() * 1000 - created) > MAX_PAIR_AGE_HOURS * 3600 * 1
        return False, "trop ancien"
    if REQUIRE_PROFILE:
        has_img = bool(info.get("imageUrl"))
        has_links = bool(info.get("websites") or info.get("socials"))
        if not (has_img and has_links):
            return False, "profil incomplet"
    return True, ""


def fmt_usd(n: float) -> str:
    return f"${n:,.0f}"


def build_message(pair: dict) -> str:
    base = pair.get("baseToken") or {}
    info = pair.get("info") or {}
    liq = num((pair.get("liquidity") or {}).get("usd"))
    vol = num((pair.get("volume") or {}).get("h24"))
    mcap = num(pair.get("marketCap")) or num(pair.get("fdv"))
    chg = (pair.get("priceChange") or {})
    age_min = (time.time() * 1000 - num(pair.get("pairCreatedAt"))) / 60000
    links = []
    for w in info.get("websites") or []:
        links.append(f'<a href="{w.get("url")}">Site</a>')
    for s in info.get("socials") or []:
        links.append(f'<a href="{s.get("url")}">{s.get("type", "lien").capitalize
    return (
        f"🚨 <b>CALL — {base.get('name')} (${base.get('symbol')})</b>\n"
        f"⛓️ {pair.get('chainId')} | DEX : {pair.get('dexId')}\n\n"
        f"💰 MCap : {fmt_usd(mcap)}\n"
        f"💧 Liquidité : {fmt_usd(liq)}\n"
        f"📊 Volume 24h : {fmt_usd(vol)}\n"
        f"📈 5m : {chg.get('m5', 'n/a')}% | 1h : {chg.get('h1', 'n/a')}%\n"
        f"⏱️ Âge : {age_min:.0f} min\n\n"
```

```
        f"🔗 {' · '.join(links) if links else 'aucun lien'}\n"
        f"📋 <code>{base.get('address')}</code>\n"
        f'👉 <a href="{pair.get("url")}">Ouvrir sur DexScreener</a>'
    )


async def send_telegram(session: aiohttp.ClientSession, text: str):
    if not TG_TOKEN or not TG_CHAT:
        log.error("TELEGRAM_TOKEN / TELEGRAM_CHAT_ID manquants !\n%s", text)
        return
    url = f"https://api.telegram.org/bot{TG_TOKEN}/sendMessage"
    payload = {
        "chat_id": TG_CHAT,
        "text": text,
        "parse_mode": "HTML",
        "disable_web_page_preview": False,
    }
    try:
        async with session.post(url, json=payload, timeout=aiohttp.ClientTimeout(
            if r.status != 200:
                log.warning("Telegram HTTP %s : %s", r.status, await r.text())
    except Exception as e:
        log.warning("Telegram erreur : %s", e)


async def check_batch(session: aiohttp.ClientSession, chain: str, addrs: list[str
    """Vérifie jusqu'à 30 tokens d'une même chaîne en un seul appel."""
    data = await get_json(session, f"{API}/tokens/v1/{chain}/{','.join(addrs)}")
    if data is None:
        return
    # Meilleure paire (liquidité max) par token
    best: dict[str, dict] = {}
    for p in data:
        addr = (p.get("baseToken") or {}).get("address", "").lower()
        liq = num((p.get("liquidity") or {}).get("usd"))
        if addr and (addr not in best or liq > num((best[addr].get("liquidity") o
            best[addr] = p

    for addr in addrs:
        key = f"{chain}:{addr.lower()}"
        pair = best.get(addr.lower())
        if key in alerted:
            watchlist.pop(key, None)
            continue
        if not pair:
            continue  # pas encore de paire indexée, on réessaiera
        ok, why = evaluate(pair)
```

```
        if ok:
            alerted.add(key)
            watchlist.pop(key, None)
            save_alerted()
            log.info("CALL %s", key)
            await send_telegram(session, build_message(pair))
        else:
            log.debug("%s refusé : %s", key, why)


async def profile_loop(session: aiohttp.ClientSession):
    """Détecte en continu les nouveaux profils DexScreener (= profil payé)."""
    while True:
        started = time.time()
        profiles = await get_json(session, f"{API}/token-profiles/latest/v1")
        if isinstance(profiles, list):
            fresh: dict[str, list[str]] = {}
            for p in profiles:
                chain = p.get("chainId", "")
                addr = p.get("tokenAddress", "")
                if chain not in CHAINS:
                    if chain and chain not in unknown_chains_logged:
                        unknown_chains_logged.add(chain)
                        log.info("Chaîne ignorée (pas dans CHAINS) : %s", chain)
                    continue
                key = f"{chain}:{addr.lower()}"
                if key in seen_profiles or key in alerted:
                    continue
                seen_profiles.add(key)
                watchlist[key] = (chain, addr, time.time())
                fresh.setdefault(chain, []).append(addr)
                log.info("Nouveau profil : %s", key)
            for chain, addrs in fresh.items():
                for i in range(0, len(addrs), 30):
                    # test immédiat, sans attendre la boucle de re-check
                    asyncio.create_task(check_batch(session, chain, addrs[i:i + 3
        await asyncio.sleep(max(0.0, PROFILE_POLL_SECONDS - (time.time() - starte


async def recheck_loop(session: aiohttp.ClientSession):
    """Re-teste les tokens profilés qui ne remplissaient pas encore les critères.
    while True:
        await asyncio.sleep(RECHECK_SECONDS)
        now = time.time()
        by_chain: dict[str, list[str]] = {}
        for key, (chain, addr, added) in list(watchlist.items()):
            if now - added > WATCH_EXPIRY_HOURS * 3600:
```

```
                watchlist.pop(key, None)
            else:
                by_chain.setdefault(chain, []).append(addr)
        for chain, addrs in by_chain.items():
            for i in range(0, len(addrs), 30):
                await check_batch(session, chain, addrs[i:i + 30])


async def main():
    load_alerted()
    log.info("Bot démarré. Chaînes : %s", ", ".join(sorted(CHAINS)))
    connector = aiohttp.TCPConnector(limit=50, ttl_dns_cache=300)
    async with aiohttp.ClientSession(connector=connector) as session:
        await send_telegram(session, "✅ Bot d'alerte démarré.")
        await asyncio.gather(profile_loop(session), recheck_loop(session))


if __name__ == "__main__":
    try:
        asyncio.run(main())
    except KeyboardInterrupt:
        pass
```